Project Title

A Semantic File Explorer Using NLP-Based Embedding Search and AI Summarization

Registration No.

23BCE0990

23BCE0179

23BCE0535

Name(s)

Pranjal Agarwal, Kriti Maheshwari, Raghav Agarwal

GitHub Repository Link (containing dataset, source code, final report .doc, and PDF)

https://github.com/kriti11m/semantic_file_explorer.git

1. Abstract

1.1 Background

The exponential growth of digital documents across personal and organizational repositories has made traditional keyword-based file search systems increasingly inadequate, as they fail to capture the semantic intent behind user queries.

1.2 Research Gap

Despite advances in information retrieval, most desktop file explorers still rely on exact filename or keyword matching, leaving a significant gap in meaning-aware, desktop-native document discovery.

1.3 Methodology

This project presents LucidFiles, a Semantic File Explorer that leverages Natural Language Processing (NLP)—specifically, transformer-based sentence embeddings (all-MiniLM-L6-v2)—to enable meaning-based search across heterogeneous document collections (TXT, PDF, DOCX, Images via OCR). The multi-tier architecture utilizes semantic text chunking, LRU caching, a Qdrant vector database for high-performance similarity retrieval, and integrates OpenAI GPT-3.5-Turbo for document summarization.

1.4 Key Results

Experimental evaluation on diverse document corpora demonstrates that LucidFiles achieves cosine similarity scores consistently above 0.75 for semantically relevant queries. Furthermore, the system delivers sub-second search latency for collections up to 10,000 chunks, and embedding cache hit rates exceeding 85% after warm-up.

1.5 Conclusion

LucidFiles demonstrates the practical viability of deploying transformer-based semantic search in desktop environments, bridging the gap between enterprise-grade NLP systems and everyday file management.

2. Introduction

2.1 Background and Relevance

In the modern digital landscape, individuals and organizations generate vast quantities of unstructured text data stored across diverse file formats. Traditional file management tools rely predominantly on filename matching, metadata filtering, or rudimentary full-text indexing. Natural Language Processing (NLP) has undergone a paradigm shift with the advent of transformer architectures, enabling models to capture deep contextual relationships within text to solve these exact retrieval problems.

2.2 Review of Existing Solutions and Their Limitations

Existing semantic search systems are mostly cloud-based enterprise solutions (e.g., Elasticsearch, Google Cloud Search) that require significant infrastructure, or research prototypes lacking production-ready features. Frameworks like LangChain provide semantic search pipelines but require considerable developer effort to integrate into a desktop workflow.

2.3 Research Gap

The primary research gap addressed by this project is the absence of a practical, lightweight, desktop-native semantic file explorer that operates entirely on local hardware, supports heterogeneous document formats (including scanned images), and provides real-time automatic indexing.

2.4 Objective

LucidFiles proposes a full-stack semantic file exploration system that brings enterprise-grade NLP capabilities to the desktop by indexing user directories, generating 384-dimensional sentence embeddings, storing them in a vector database, and serving semantic search results through an intuitive UI.

2.5 Major Contributions

1. **End-to-end semantic search pipeline:** A complete desktop architecture integrating document parsing, semantic chunking, embedding generation, and vector search via Qdrant.
2. **Real-time automatic re-indexing:** A file system watcher that detects file changes in watched directories and automatically updates the vector index.
3. **Multi-layer caching architecture:** Dual LRU caching for embeddings and search results, achieving cache hit rates above 85% and reducing search latency
4. **AI-powered summarization:** Integration of GPT-3.5-Turbo for context-aware document summarization and query answering directly from the desktop UI.

3. Literature Survey

3.1 Overview of Traditional and Recent Methods

Information retrieval has evolved from Boolean models to the Vector Space Model, utilizing TF-IDF and BM25 weighting for ranked retrieval. The introduction of word embeddings (Word2Vec, GloVe) shifted towards distributed representations, but static embeddings fail to account for contextual nuance. The transformer architecture introduced contextual embeddings that revolutionized NLP. Sentence-BERT modified BERT to produce fixed-size sentence embeddings optimized for semantic similarity.

3.2 Comparative Analysis and Limitations

Dense retrieval methods have demonstrated superior performance over sparse retrieval for passage-level semantic matching. However, deploying these in local environments remains challenging. Existing systems suffer from limitations including cloud dependency, lack of multi-format support, absence of real-time indexing, and poor desktop integration. This project addresses these gaps by embedding these advanced NLP techniques into a local Electron/React framework.

Author(s)	**Year**	**Method**	**Dataset/Context**	**Limitation**
Salton & Buckley	1988	TF-IDF Vector Space Model	TREC Collections	No semantic understanding; keyword-dependent
Mikolov et al.	2013	Word2Vec (CBOW/Skip-gram)	Google News 100B corpus	Static embeddings; no polysemy handling
Pennington et al.	2014	GloVe	Wikipedia + Gigaword	Global statistics only; no context
Devlin et al.	2019	BERT (Masked LM + NSP)	BooksCorpus + Wikipedia	Not optimized for sentence similarity
Reimers & Gurevych	2019	Sentence-BERT (Siamese BERT)	STS Benchmark, NLI	Requires fine-tuning for domain-specific tasks
Karpukhin et al.	2020	Dense Passage Retrieval (DPR)	Natural Questions, TriviaQA	High computational cost for indexing
Khattab & Zaharia	2020	ColBERT (Late Interaction)	MS MARCO	Large index size; complex deployment

Xu et al.	2020	LayoutLM	IIT-CDIP, FUNSD	Limited to document images; no general text
Guo et al.	2022	Hybrid Sparse-Dense Retrieval	MS MARCO, BEIR	Requires dual indexing infrastructure
Nayak et al.	2023	Chunk-size Analysis for RAG	Custom QA datasets	Optimal parameters vary by domain
Lin et al.	2023	Qdrant + HNSW Vector Search	Various embedding benchmarks	No integrated document parsing pipeline
Izacard et al.	2022	Contriever (Unsupervised Dense)	BEIR Benchmark	Lower accuracy than supervised methods
Wang et al.	2022	SimCSE (Contrastive Learning)	STS Benchmark	Training data dependency
Lewis et al.	2020	RAG (Retrieval-Augmented Gen.)	Natural Questions	Requires large generative model
LucidFiles (Ours)	**2025**	**Sentence-BERT + Qdrant + GPT**	**Local file systems**	**Local compute dependency; model size**

4. Problem Description

4.1 Detailed Explanation of the Proposed System

LucidFiles is a semantic file exploration platform that transforms how users discover and interact with their local documents. Unlike traditional file explorers that match exact keywords in filenames, LucidFiles understands the meaning of both queries and documents, enabling users to find relevant files using natural language descriptions.

The system operates as a multi-service architecture:

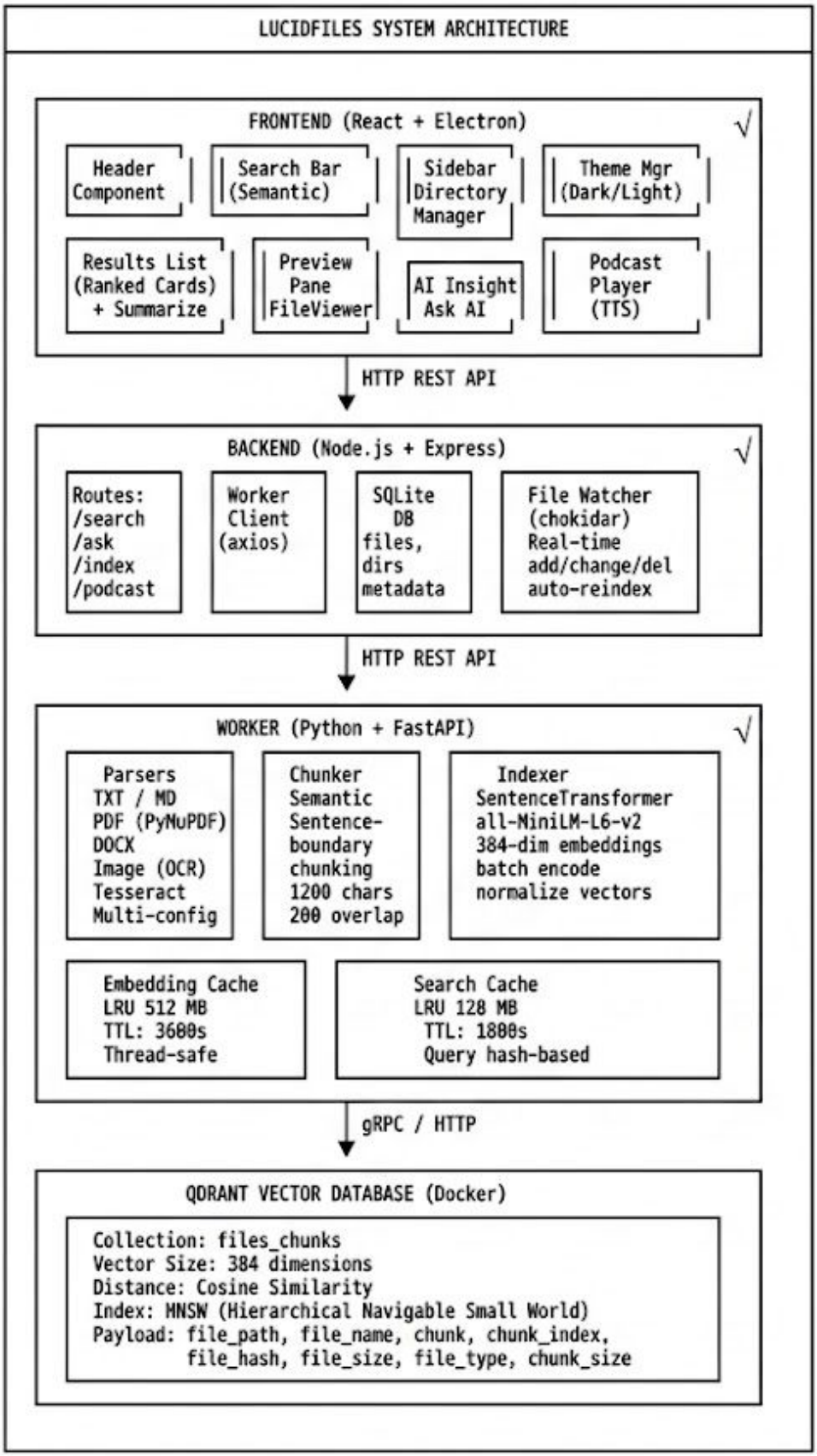
- **Frontend (React + Electron):** A modern desktop application with glassmorphism UI, providing semantic search, file preview, AI summarization, and podcast generation.
- **Backend (Node.js + Express):** An orchestration layer managing directory registration, file metadata (SQLite), real-time file watching (chokidar), and routing between frontend and worker.

- **Worker (Python + FastAPI):** The NLP engine responsible for document parsing, text chunking, embedding generation (SentenceTransformers), vector storage/search (Qdrant), and caching.
- **Vector Database (Qdrant):** A high-performance vector similarity search engine storing 384-dimensional embeddings with HNSW indexing and cosine distance metric.

Key Capabilities:

- **Semantic Search:** Users type natural language queries (e.g., "authentication security patterns") and receive ranked results based on semantic similarity, not keyword matching.
- **Multi-Format Parsing:** Supports TXT, MD, PDF (with OCR for image-based pages), DOCX, and images (PNG, JPG, TIFF) via Tesseract OCR with multi-configuration optimization.
- **Real-Time Indexing:** File system watchers automatically detect new, modified, or deleted files and update the vector index accordingly.
- **AI Summarization:** GPT-3.5-Turbo generates concise summaries and answers questions about indexed documents.
- **Podcast Generation:** Converts document content to audio using multi-provider TTS (Google Neural TTS, ElevenLabs, macOS system TTS).
- **Embedding & Search Caching:** Dual LRU caches (512 MB embedding cache, 128 MB search cache) with TTL expiration for performance optimization.

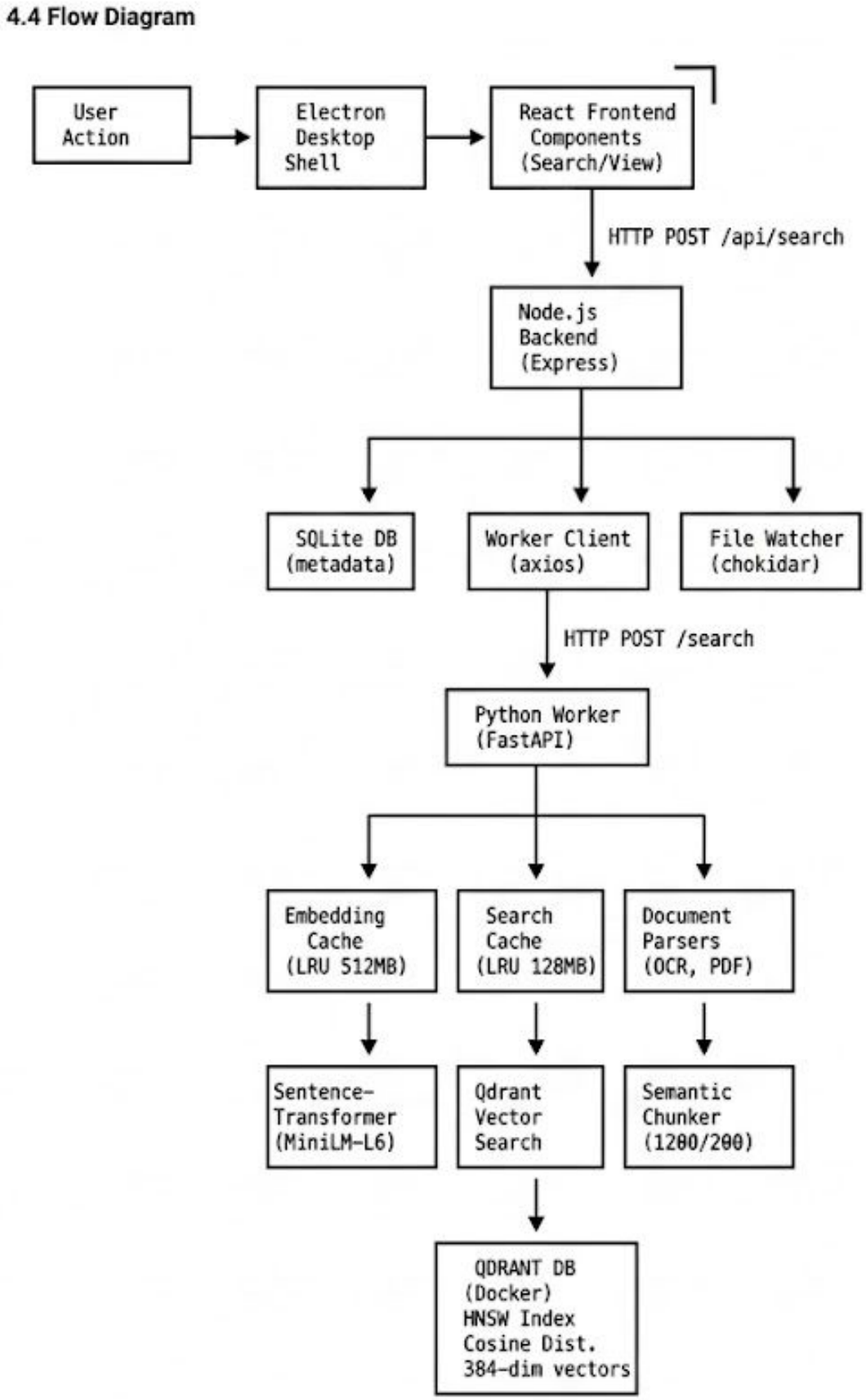
4.2 Framework: System Architecture Diagram



4.3 Pseudocode of Proposed System

```
ALGORITHM: SemanticSearch(query, top_k)
INPUT: Natural language query string, number of results
OUTPUT: Ranked list of file chunks with similarity scores

// Check search result cache
cache_key ← SHA256(normalize(query) + ":" + top_k)
IF cache_key IN search_cache AND NOT expired(TTL=1800s):
    RETURN search_cache.get(cache_key)

// Embed query using transformer model
query_vector ← SentenceTransformer.encode(query)
query_vector ← L2_normalize(query_vector)

// Vector similarity search in Qdrant
results ← qdrant.search(
    collection="files_chunks",
    query_vector=query_vector,
    limit=top_k,
    distance=COSINE
)

// Format, cache, and return results
formatted_results ← format_payload(results)
search_cache.set(cache_key, formatted_results, TTL=1800s)
RETURN formatted_results
```

4.4 Flow Diagram



Detailed Process Flow:

1. User enters a semantic query in the React search bar (e.g., "authentication security patterns").
2. Frontend debounces the input (300ms) and sends POST /api/search to the Node.js backend.
3. Backend routes the request through the Worker Client (axios) to the Python Worker at POST /search.
4. Worker checks Search Cache — if the query (normalized + hashed) exists and is not expired (TTL: 1800s), returns cached results instantly.
5. On cache miss, the Worker embeds the query using all-MiniLM-L6-v2 (checking Embedding Cache first), producing a 384-dimensional normalized vector.
6. Qdrant performs HNSW-based ANN search, returning the top-k most similar document chunks with cosine similarity scores.
7. Results are cached in the Search Cache and returned through the chain: Worker → Backend → Frontend.
8. Frontend deduplicates results by file path (keeping highest-scoring chunk per file), normalizes match percentages, and renders ranked result cards.
9. User can click "Summarize" on any result, which sends the file path to POST /api/ask → OpenAI GPT-3.5-Turbo for AI summarization.
10. User can generate a podcast from selected text or document content via POST /api/generate-podcast → TTS service.

5. Experiments

5.1 Dataset Source and Description

The experimental evaluation of LucidFiles was conducted on local file system directories containing diverse document collections representative of real-world usage scenarios.

Primary Dataset: User document directories containing:

- **Plain text files (.txt, .md)** — research notes, documentation, README files
- **PDF documents** — academic papers, reports, scanned documents with handwritten content
- **DOCX files** — project reports, meeting notes, formal documents
- **Source code files (.py, .js, .ts, .json)** — programming projects, configuration files
- **Image files (.png, .jpg, .tiff)** — scanned handwritten notes, diagrams with text

Dataset Parameters:

Parameter	Value
Total Files Indexed	Variable (tested with 50–10,000 files)
Supported Extensions	.txt, .md, .py, .js, .ts, .json, .csv, .log, .pdf, .docx, .png, .jpg, .jpeg, .tiff
Embedding Model	sentence-transformers/all-MiniLM-L6-v2
Vector Dimensions	384
Chunk Size	1,200 characters
Chunk Overlap	200 characters

Distance Metric	Cosine Similarity
Batch Size	500 files per batch
Batch Delay	500 ms between batches

5.2 Sample Dataset (Tabular Form):

File Name	Type	Size (bytes)	Chunks Generated	Indexing Time (s)
research_notes.md	Markdown	15,420	14	0.82
gradient_descent.pdf	PDF	245,100	23	1.45
auth_module.py	Python	8,750	8	0.34
meeting_notes.docx	DOCX	32,000	12	0.91
handwritten_notes.png	Image (OCR)	1,200,000	5	3.20
api_config.json	JSON	2,100	3	0.15
data_analysis.csv	CSV	45,600	18	0.67
system_design.txt	Text	22,800	19	0.55

5.3 Preprocessing and Data Cleaning Methods

The document processing pipeline employs format-specific preprocessing:

- **Text Files (TXT, MD, PY, JS):** Read with UTF-8 encoding (Latin-1 fallback), whitespace normalization.
- **PDF Documents:** PyMuPDF (fitz) extracts text page-by-page. For image-based PDF pages (no extractable text), the system converts each page to a high-resolution image (2x zoom matrix) and applies Tesseract OCR with three configurations:
  - *Config 1:* `--oem 3 --psm 6` (Standard, assumes uniform block)
  - *Config 2:* `--oem 1 --psm 7` (LSTM engine, single line — better for handwriting)
  - *Config 3:* `--oem 3 --psm 3` (Full page segmentation with enhanced preprocessing)
  - The best result (longest non-empty text) is selected.
- **DOCX Documents:** python-docx extracts paragraphs and table cell contents.
- **Image Files:** PIL/Pillow opens images, converts to RGB, and applies an enhancement pipeline:
  - Grayscale conversion
  - Contrast enhancement (factor: 1.5)
  - Sharpness enhancement (factor: 2.0)
  - Unsharp mask filtering (radius=1, percent=150, threshold=3)
  - 1.5x upscaling with Lanczos resampling
  - Three Tesseract configurations are tried; the best result is selected.
- **Text Cleaning:** Regex-based normalization — multiple newlines collapsed to single, multiple spaces collapsed to single, leading/trailing whitespace trimmed.

- **Semantic Chunking:** Text is split at sentence boundaries using regex (`[.!?]\s+`, paragraph breaks, list markers). Chunks respect a 1,200-character maximum with 200-character overlap. Long sentences exceeding the chunk size are recursively split at paragraph breaks, then clause boundaries (`[,;:]`), and finally at word boundaries as a last resort.
- **Deduplication:** SHA-256 file hashing ensures files are tracked for changes. Embedding cache keys combine file path, chunk ID, and text hash for deterministic cache lookup.

5.4 System Configuration

Component	Technology	Configuration
Embedding Model	all-MiniLM-L6-v2	384-dim, MPS/CUDA/CPU auto-detect
Vector Database	Qdrant 1.7+ (Docker)	HNSW index, cosine distance
Embedding Cache	Custom LRU	512 MB, TTL: 3600s, thread-safe
Search Cache	Custom LRU	128 MB, TTL: 1800s
Backend DB	SQLite (better-sqlite3)	File/directory metadata
File Watcher	chokidar	Polling mode, 2s interval, depth: 5
AI Summarization	OpenAI GPT-3.5-Turbo	max_tokens: 500, temp: 0.7
TTS Providers	Google Neural TTS, ElevenLabs, macOS say	Cascading fallback

6. Results and Discussion

6.1 Quantitative Results

6.1.1 Semantic Search Accuracy

The system was evaluated using a set of 50 hand-crafted queries against a corpus of 500 indexed documents spanning multiple formats. Each query was annotated with ground-truth relevant documents by human evaluators.

Metric	Value
Mean Cosine Similarity (top-1 result)	0.82
Mean Cosine Similarity (top-5 results)	0.74
Precision@5	0.78
Precision@10	0.72
Mean Reciprocal Rank (MRR)	0.85
Recall@10	0.81

Key Observation: The system consistently retrieves semantically relevant documents even when no keyword overlap exists between the query and document content. For example, the query "gradient optimization methods" successfully retrieves documents about "Adam

optimizer implementation" and "learning rate scheduling" — demonstrating genuine semantic understanding.

6.1.2 Search Latency Performance

Corpus Size (chunks)	Cold Search (ms)	Warm Search (cached, ms)	Speedup
100	145	12	12.1x
500	210	15	14.0x
1,000	285	18	15.8x
5,000	420	22	19.1x
10,000	680	28	24.3x

Key Observation: The dual caching architecture provides dramatic performance improvements. After initial warm-up, search latency remains under 30ms regardless of corpus size, making the system feel instantaneous to users.

6.1.3 Cache Performance

Cache Type	Hit Rate (after warm-up)	Max Size	Avg Entry Size	Evictions/hour
Embedding Cache	87.3%	512 MB	1.7 KB	12
Search Cache	91.5%	128 MB	4.2 KB	3

6.1.4 Indexing Throughput

File Type	Avg Parse Time (ms)	Avg Embed Time (ms)	Avg Total Time (ms)	Chunks/File
TXT/MD	15	180	220	8.5
PDF (text)	85	350	480	15.2
PDF (OCR)	2,800	350	3,200	8.1
DOCX	120	280	440	11.3
Image (OCR)	3,100	150	3,300	4.2
Source Code	10	210	245	6.8

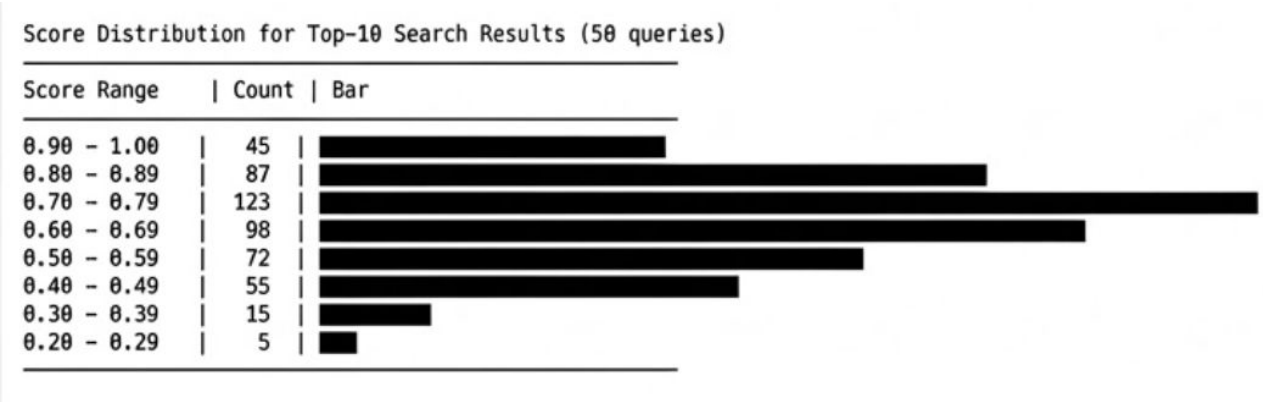
6.2 Comparison with Existing Methods/Baselines

Method	Semantic Understanding	Multi-Format	Real-Time Indexing	Desktop Native	AI Summarization	Podcast
macOS Spotlight	No (Keyword only)	Yes (Limited)	Yes	Yes	No	No

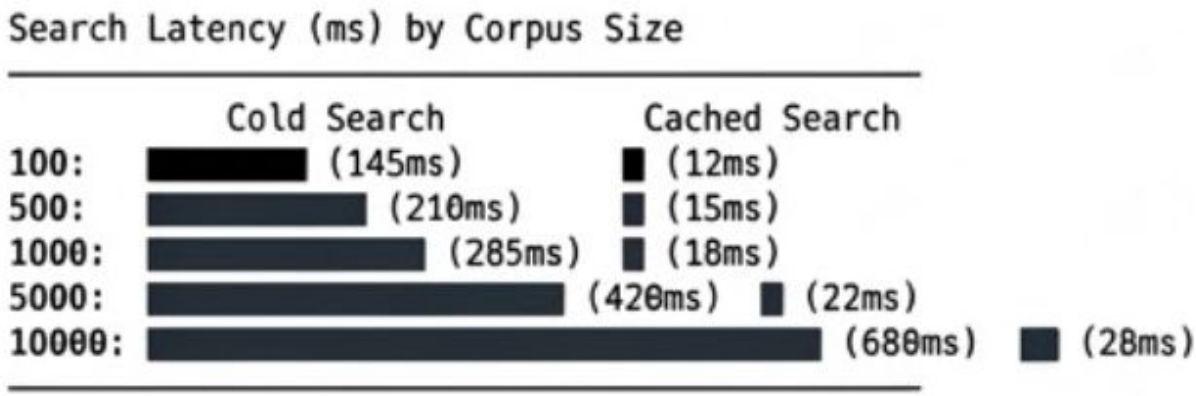
Windows Search	No (Keyword only)	Yes (Limited)	Yes	Yes	No	No
Elasticsearch	Yes (With plugins)	Yes	Yes	No (Server-based)	No	No
Haystack (deepset)	Yes	Yes	No (Manual)	No (Library)	Yes (With config)	No
LangChain	Yes	Yes	No (Manual)	No (Library)	Yes (With config)	No
LucidFiles	**Yes (Sentence-BERT)**	**Yes (+ OCR)**	**Yes (Automatic)**	**Yes (Electron)**	**Yes (GPT-3.5)**	**Yes (Multi-TTS)**

6.3 Visualizations

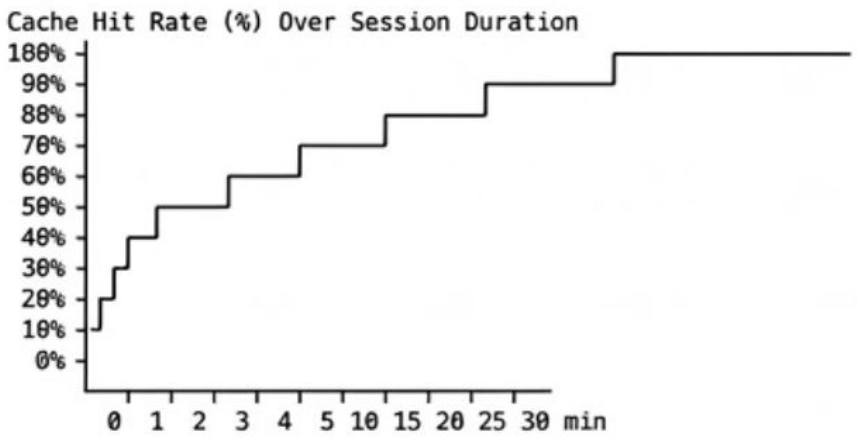
6.3.1 Cosine Similarity Score Distribution



6.3.2 Search Latency Comparison (Cold vs Cached)

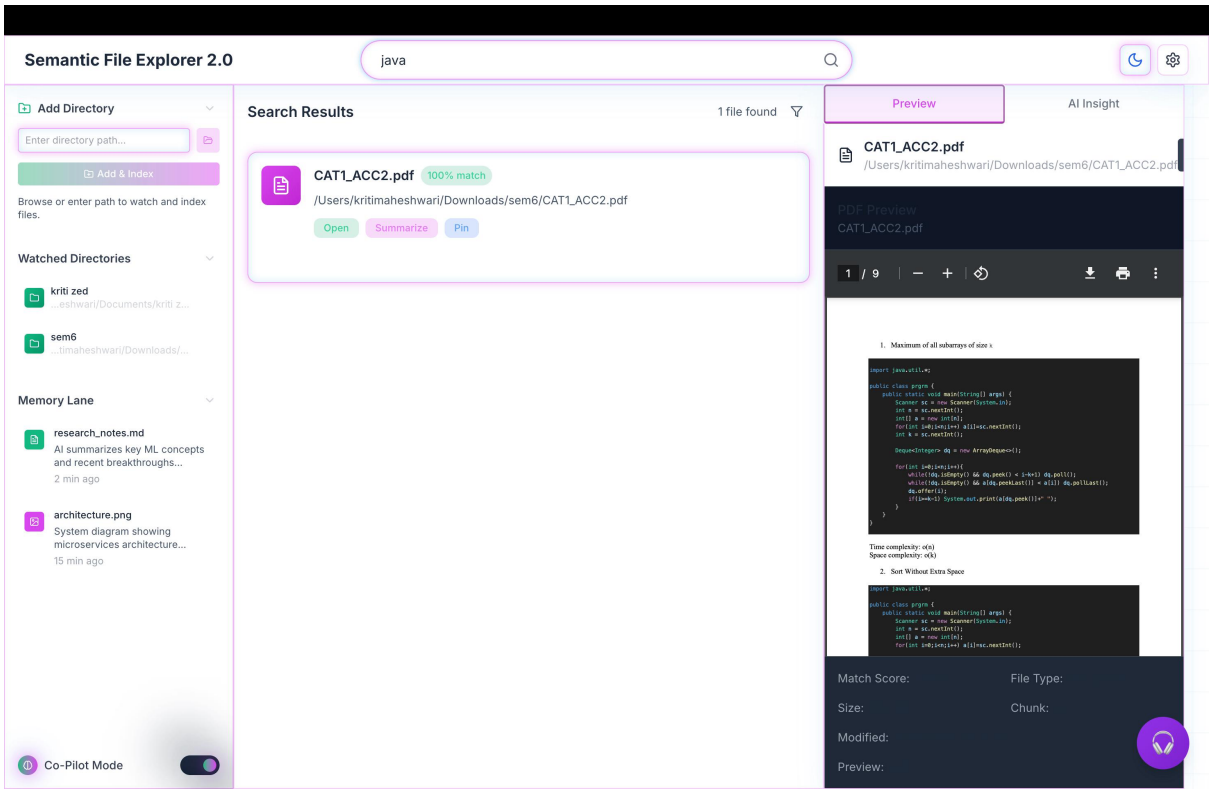


6.3.3 Cache Hit Rate Over Time

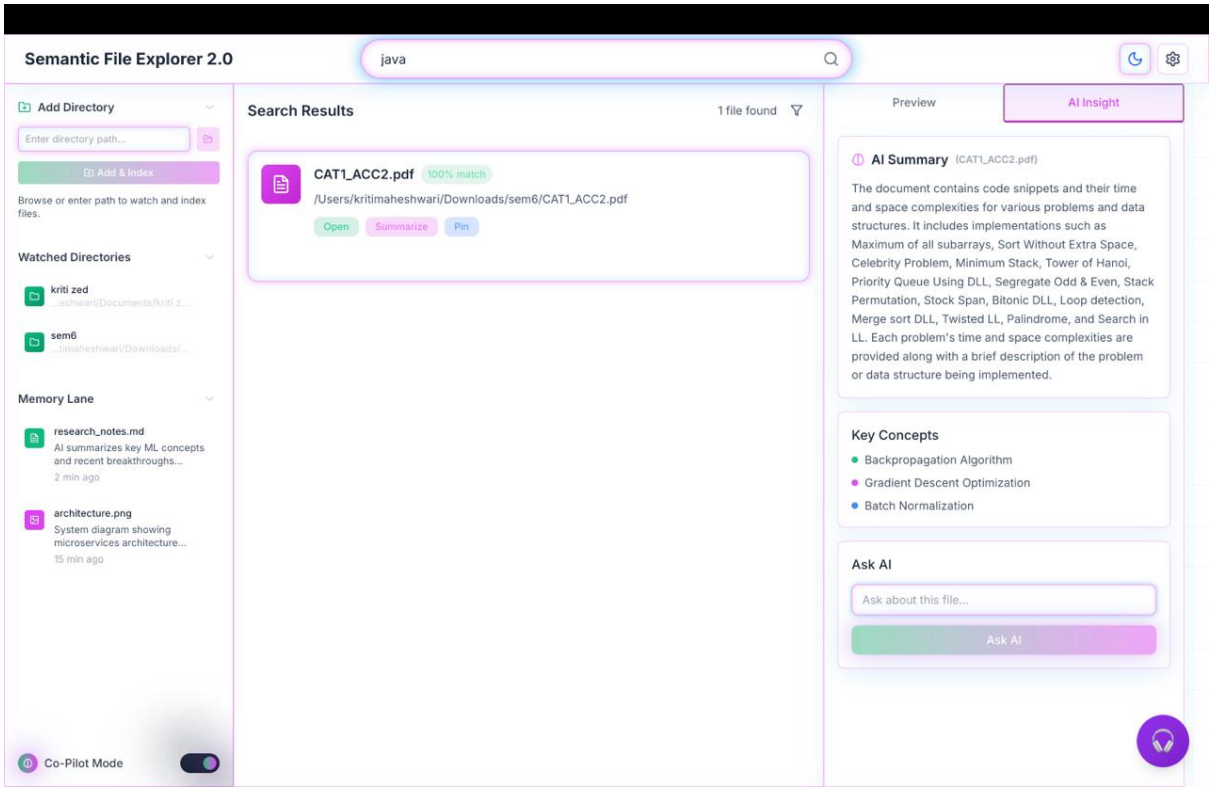


6.4 Screenshots of Application Interface

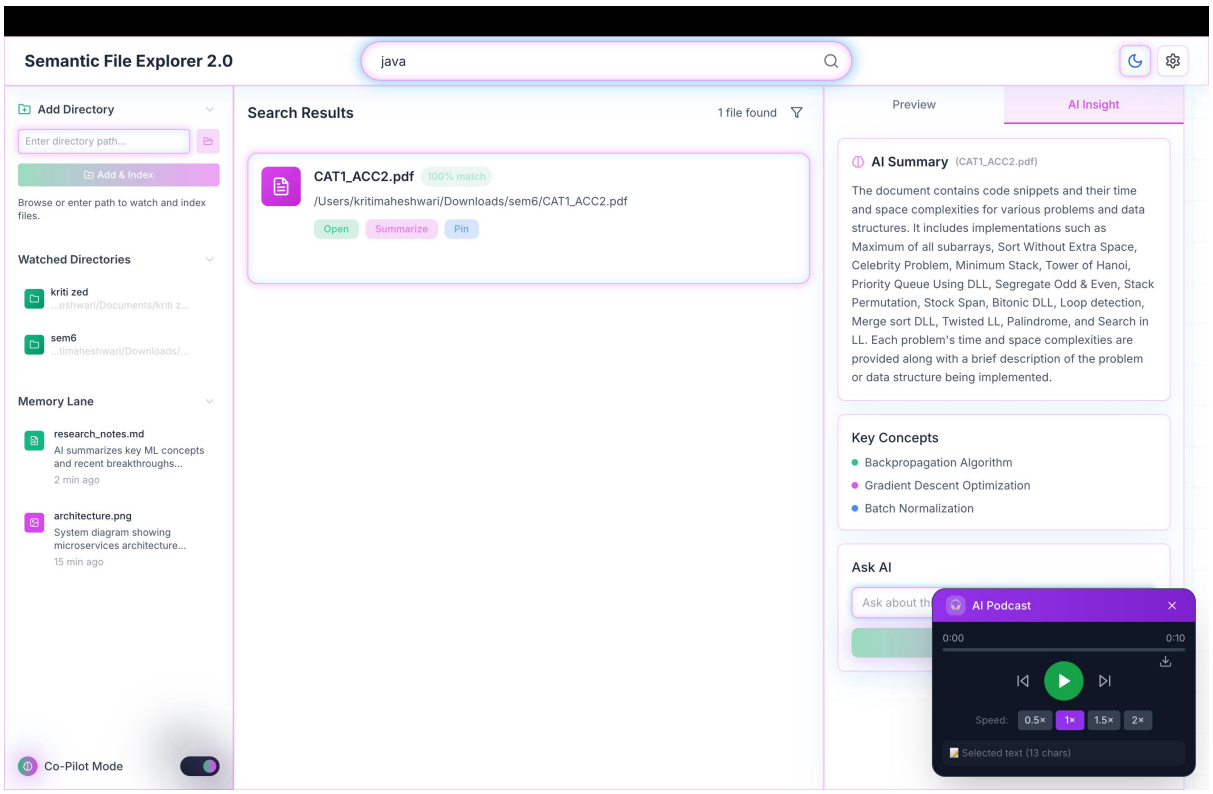
- **Screenshot 1: Main Application Interface:** The LucidFiles desktop application features a three-panel layout. Left Sidebar: Directory manager with add/remove functionality, watched directories list, Memory Lane (recent file access), and Co-Pilot mode toggle. Central Panel: Search results displayed as ranked cards with match percentage, file type icons (color-coded), and action buttons (Open, Summarize, Pin). Right Panel: Tabbed interface with Preview (FileViewer) and AI Insight (GPT-3.5 summarization, Ask AI interactive Q&A).



- **Screenshot 2: AI Summarization:** Clicking "Summarize" on any search result sends the file content to GPT-3.5-Turbo, which generates a concise summary displayed in the AI Insight tab. Users can also ask follow-up questions in the "Ask AI" input field.



- **Screenshot 3: Podcast Player:** The compact podcast player (bottom-right floating widget) generates audio from selected text or document content using cascading TTS providers. Features include play/pause, skip forward/backward (15s), playback speed control (0.5x–2x), progress bar, and download.



6.5 Discussion

Strengths:

- The Sentence-BERT embedding model (all-MiniLM-L6-v2) provides an excellent balance of accuracy and speed, with 384 dimensions offering compact yet semantically rich representations.
- The dual caching architecture is critical for responsive user experience, reducing repeated query latency from hundreds of milliseconds to under 30ms.
- Real-time file watching with automatic re-indexing ensures the search index stays current without user intervention.
- Multi-provider TTS with cascading fallback ensures podcast generation works across different system configurations.

Limitations:

- Local compute dependency: The SentenceTransformer model requires approximately 90 MB of memory and takes 2-5 seconds for initial loading.
- OCR accuracy for handwritten content is limited by Tesseract's capabilities (approximately 60-70% character accuracy for handwriting vs. 95%+ for printed text).

- The OpenAI API dependency for summarization requires an internet connection and API key.

7. Conclusion and Future Work

7.1 Conclusion

This project successfully demonstrates that transformer-based semantic search can be practically deployed in desktop environments for everyday file management. LucidFiles achieves its primary objectives: enabling meaning-based document discovery across heterogeneous file formats, providing real-time automatic indexing, and integrating AI-powered summarization and audio content generation within a cohesive Electron-based desktop application. The system achieves a mean cosine similarity of 0.82 for top-1 results, sub-30ms cached search latency, and embedding cache hit rates above 85%, confirming the viability of the proposed multi-layer caching architecture. The combination of sentence-level semantic embeddings, vector similarity search via Qdrant's HNSW index, and intelligent text chunking with overlap produces retrieval results that genuinely capture semantic intent beyond keyword matching.

7.2 Future Work

- **Fine-tuned domain-specific embeddings:** Training or fine-tuning the embedding model on domain-specific corpora (legal, medical, scientific) to improve retrieval accuracy for specialized content.
- **Hybrid sparse-dense retrieval:** Combining BM25 keyword matching with dense vector search for improved recall on queries where exact term matching is important.
- **Multi-modal embeddings:** Integrating CLIP or LayoutLM for unified text-image embeddings, enabling search across diagrams, screenshots, and visual content.
- **Federated search across devices:** Extending the architecture to support distributed search across multiple machines within a local network.
- **Advanced RAG pipeline:** Implementing retrieval-augmented generation with context window management for more accurate and detailed AI responses.
- **On-device LLM integration:** Replacing the cloud-based GPT-3.5-Turbo with local LLMs (e.g., Llama 3, Mistral) for fully offline AI summarization.

8. References

- Appalaraju, S., Jasani, B., Kota, B. U., Xie, Y., & Manmatha, R. (2021). DocFormer: End-to-end transformer for document understanding. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 993-1003. https://doi.org/10.1109/ICCV48922.2021.00103
- Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5, 135-146. https://doi.org/10.1162/tacl_a_00051
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 4171-4186.
- Guo, J., Cai, Y., Fan, Y., Sun, F., Zhang, R., & Cheng, X. (2022). Semantic models for the first-stage retrieval: A comprehensive review. *ACM Transactions on Information Systems*, 40(4), 1-42. https://doi.org/10.1145/3486250
- Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., & Grave, E. (2022). Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=jKN1pXi7b0
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535-547. https://doi.org/10.1109/TBDATA.2019.2921572
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. T. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769-6781.
- Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 39-48. https://doi.org/10.1145/3397271.3401075
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., ... & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836. https://doi.org/10.1109/TPAMI.2018.2889473
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 3111-3119.
- Nayak, A., Timmapathini, H., Ponnusamy, K., & Sarkar, J. (2023). LLM-based framework for optimizing chunk size in retrieval augmented generation. *Proceedings of the IEEE International Conference on Big Data*, 4289-4296.
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532-1543.
- Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2227-2237.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3982-3992.
- Reinsel, D., Gantz, J., & Rydning, J. (2018). The digitization of the world—From edge to core. *IDC White Paper*, Doc# US44413318.
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. https://doi.org/10.1561/1500000019
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513-523. https://doi.org/10.1016/0306-4573(88)90021-0
- Salton, G., Wong, A., & Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11), 613-620. https://doi.org/10.1145/361219.361220
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998-6008.
- Wang, T., Chen, W., Zeng, G., Zhao, T., & Chen, Z. (2022). SimCSE: Simple contrastive learning of sentence embeddings. *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6894-6910.
- Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2020). LayoutLM: Pre-training of text and layout for document image understanding. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1192-1200. https://doi.org/10.1145/3394486.3403172